# Exercício C

**ALUNO:Ruan Silva Carvalho**

Exercício C1 — Tamanho total do pacote

#include <stdio.h> #include <limits.h>

#define TAM_CABECALHO 20 /* cabecalho fixo da camada de rede, em bytes */

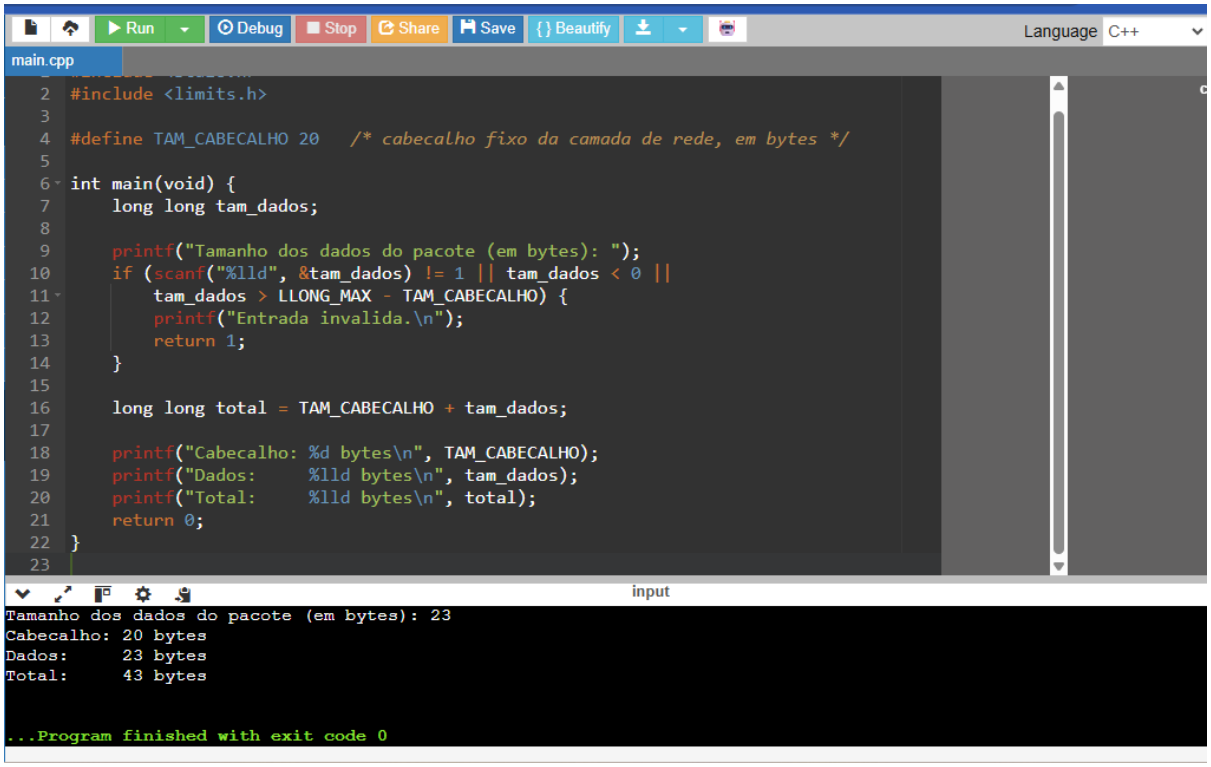
int main(void) { long long tam_dados;

```
printf("Tamanho dos dados do pacote (em bytes): ");
if (scanf("%lld", &tam_dados) != 1 || tam_dados < 0
||
    tam_dados > LLONG_MAX - TAM_CABECALHO) {
    printf("Entrada invalida.\n");
    return 1;
}

long long total = TAM_CABECALHO + tam_dados;

printf("Cabecalho: %d bytes\n", TAM_CABECALHO);
printf("Dados:     %lld bytes\n", tam_dados);
printf("Total:     %lld bytes\n", total);
return 0;


}
```




Exercício C2 — Formatador de endereço físico

#include <stdio.h>

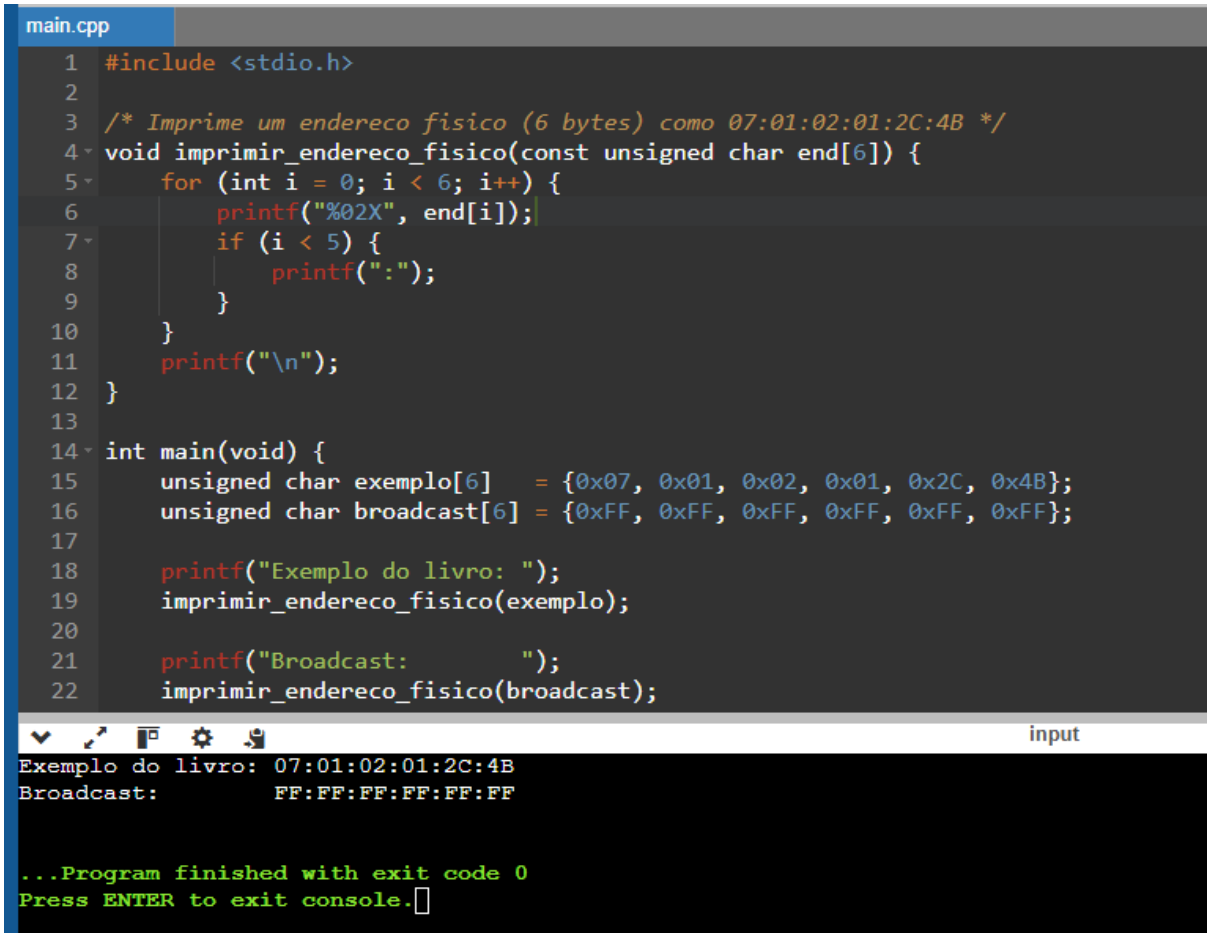
/* Imprime um endereco fisico (6 bytes) como 07:01:02:01:2C:4B */ void imprimir_endereco_fisico(const unsigned char end[6]) { for (int i = 0; i < 6; i++) { printf("%02X", end[i]); if (i < 5) { printf(":"); } } printf("\n"); }

int main(void) { unsigned char exemplo[6] = {0x07, 0x01, 0x02, 0x01, 0x2C, 0x4B}; unsigned char broadcast[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

```
printf("Exemplo do livro: ");
imprimir_endereco_fisico(exemplo);

printf("Broadcast:        ");
imprimir_endereco_fisico(broadcast);
return 0;
```

}




Exercício C3 — Classificador de camadas OSI

#include <stdio.h>

typedef enum { FISICA, ENLACE, REDE, TRANSPORTE, SESSAO, APRESENTACAO, APLICACAO } CamadaOSI;

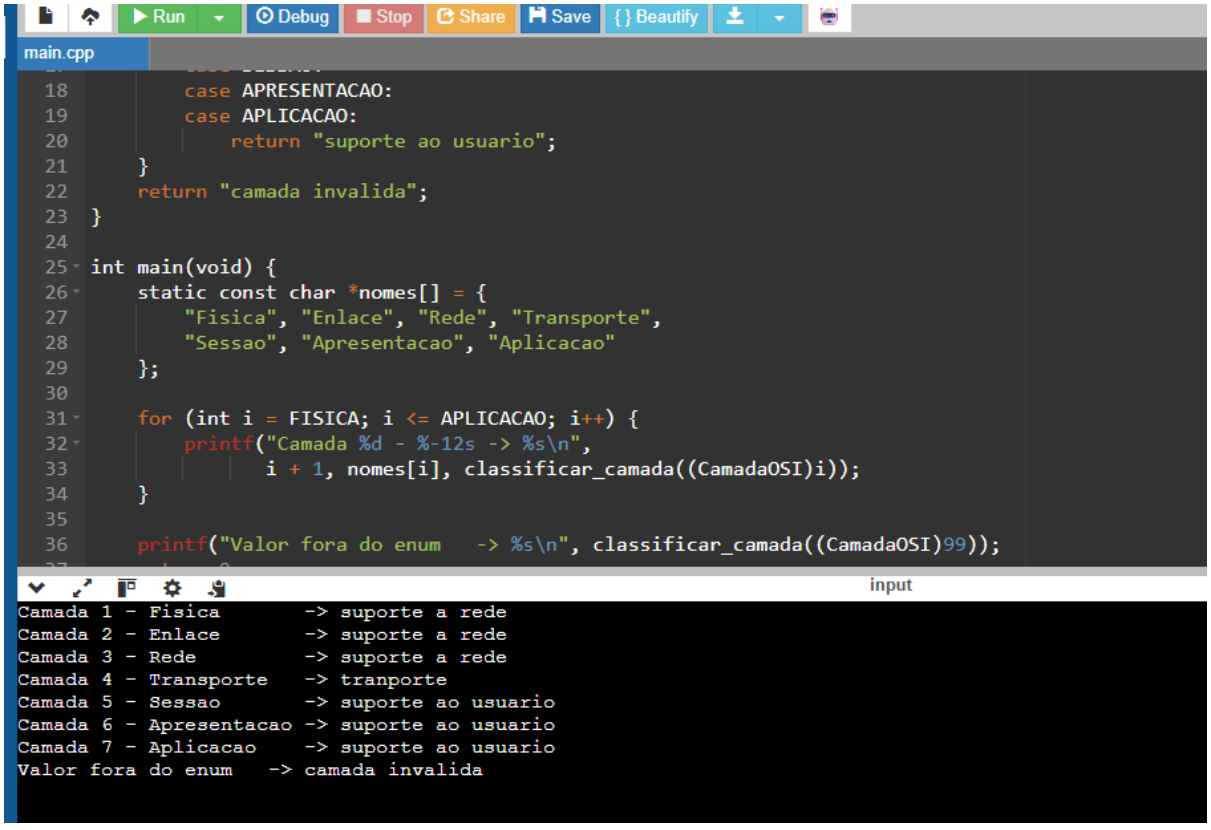
/* Camadas 1,2,3 = suporte a rede | 4 = transporte | 5,6,7 = suporte ao usuario */ const char *classificar_camada(CamadaOSI c) { switch (c) { case FISICA: case ENLACE: case REDE: return "suporte a rede"; case TRANSPORTE: return "transporte"; case SESSAO: case APRESENTACAO: case APLICACAO: return "suporte ao usuario"; } return "camada invalida"; }

int main(void) { static const char *nomes[] = { "Fisica", "Enlace", "Rede", "Transporte", "Sessao", "Apresentacao", "Aplicacao" };

```
for (int i = FISICA; i <= APLICACAO; i++) {
    printf("Camada %d - %-12s -> %s\n",
           i + 1, nomes[i],
classificar_camada((CamadaOSI)i));
}

printf("Valor fora do enum   -> %s\n",
classificar_camada((CamadaOSI)99));
return 0;


}
```




Exercício C4 — Corrigir a struct de cabeçalho

#include <stdio.h> #include <string.h> #include <stddef.h>

/* CORRIGIDO: o cabecalho vem ANTES dos dados (H | Dados), como no encapsulamento real. */ typedef struct { char cabecalho[16]; char dados[128]; } Pacote;

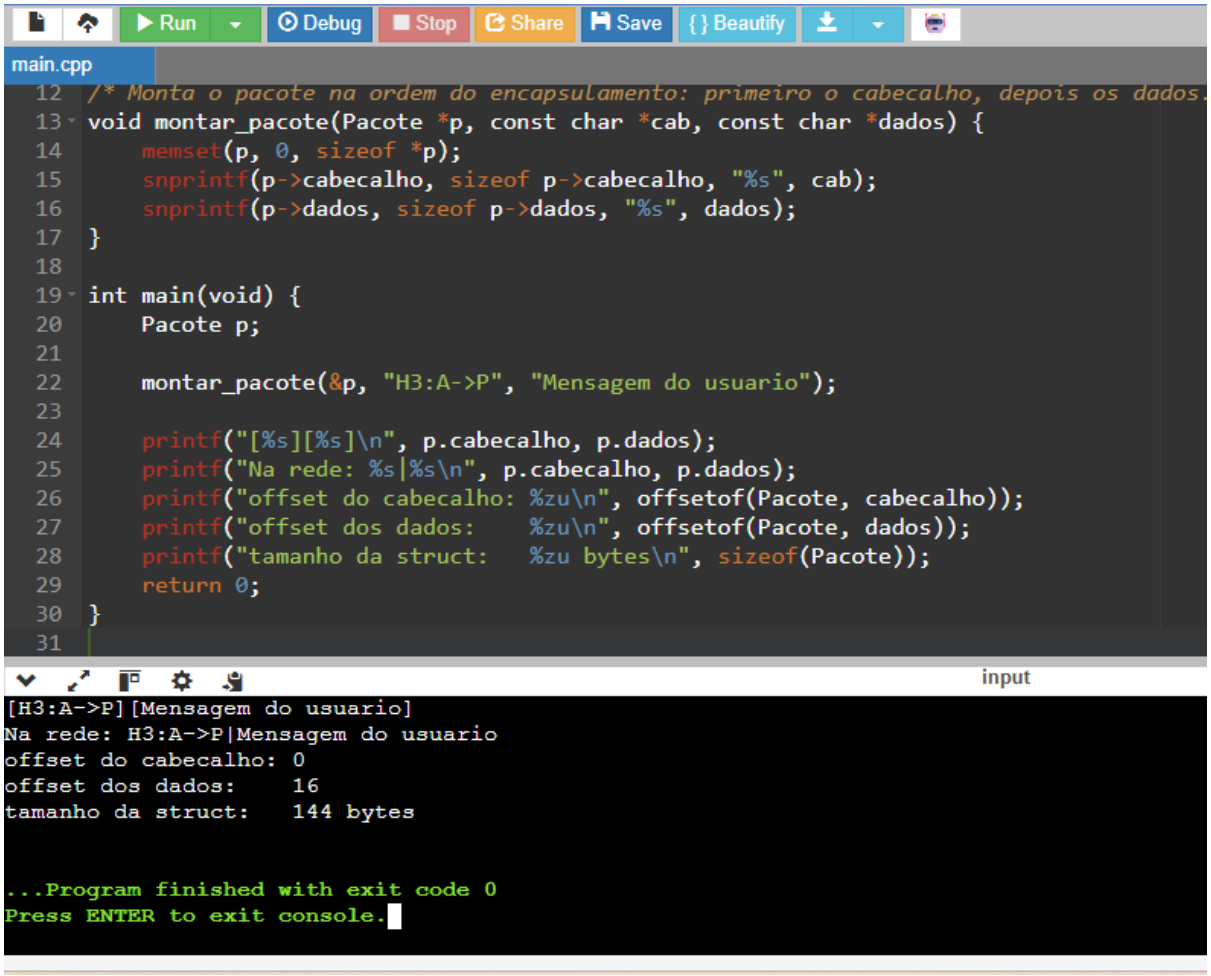
/* Monta o pacote na ordem do encapsulamento: primeiro o cabecalho, depois os dados. */ void montar_pacote(Pacote *p, const char *cab, const char *dados) { memset(p, 0, sizeof *p); snprintf(p->cabecalho, sizeof p->cabecalho, "%s", cab); snprintf(p->dados, sizeof p->dados, "%s", dados); }

int main(void) { Pacote p;

```
montar_pacote(&p, "H3:A->P", "Mensagem do
usuario");

printf("[%s][%s]\n", p.cabecalho, p.dados);
printf("Na rede: %s|%s\n", p.cabecalho, p.dados);
printf("offset do cabecalho: %zu\n",
offsetof(Pacote, cabecalho));
printf("offset dos dados:    %zu\n",
offsetof(Pacote, dados));
printf("tamanho da struct:   %zu bytes\n",
sizeof(Pacote));
return 0;


}
```




Exercício C5 — Segmentação e remontagem

#include <stdio.h> #include <string.h> #include <stdlib.h>

#define MAX_SEGS 64 #define TAM_MAX_SEG 32

typedef struct { int seq; /* numero de sequencia */ *char dados[TAM_MAX_SEG + 1]; /* pedaco da mensagem */ } Segmento;

/* Divide msg em segmentos numerados de tam_seg bytes. Retorna o numero de segmentos, ou -1 em caso de erro. */ int segmentar(const char *msg, int tam_seg, Segmento segs[], int max) { if (tam_seg <= 0 || tam_seg > TAM_MAX_SEG) { return -1; } size_t n = strlen(msg); int num = (int)((n + (size_t)tam_seg - 1) / (size_t)tam_seg); if (num > max) { return -1; } for (int i = 0; i < num;

i++) { memset(&segs[i], 0, sizeof segs[i]); segs[i].seq = i + 1; strncpy(segs[i].dados, msg + (size_t)i * (size_t)tam_seg, (size_t)tam_seg); } return num; }

static int comparar_seq(const void *a, const void *b) { const Segmento *x = a, *y = b; return (x->seq > y->seq) - (x->seq < y->seq); }

/* Junta os segmentos na ordem correta (pelo numero de sequencia). Retorna 0 se deu certo, -1 se faltou espaco, -2 se faltou algum segmento. */ int remontar(const Segmento segs[], int n, char *saida, size_t cap) { Segmento tmp[MAX_SEGS];

```
if (n < 0 || n > MAX_SEGS || cap == 0) {
    return -1;
}
memcpy(tmp, segs, (size_t)n * sizeof(Segmento));
qsort(tmp, (size_t)n, sizeof(Segmento),
comparar_seq);

size_t len = 0;
saida[0] = '\0';
for (int i = 0; i < n; i++) {
    if (tmp[i].seq != i + 1) {
        return -2;
    }
    size_t l = strlen(tmp[i].dados);
    if (len + l + 1 > cap) {
        return -1;
    }
    memcpy(saida + len, tmp[i].dados, l + 1);
    len += l;
}
```

```
return 0;


}
```

int main(void) { const char *msg = "COMUNICACAODEDADOS"; Segmento segs[MAX_SEGS]; char saida[256];

```
int n = segmentar(msg, 8, segs, MAX_SEGS);
if (n < 0) {
    printf("Erro ao segmentar.\n");
    return 1;
}

printf("Mensagem original: %s\n\n", msg);
for (int i = 0; i < n; i++) {
    printf("Segmento %d: %s\n", segs[i].seq,
segs[i].dados);
}

/* simula chegada fora de ordem: troca o primeiro
com o ultimo */
Segmento aux = segs[0];
segs[0] = segs[n - 1];
segs[n - 1] = aux;

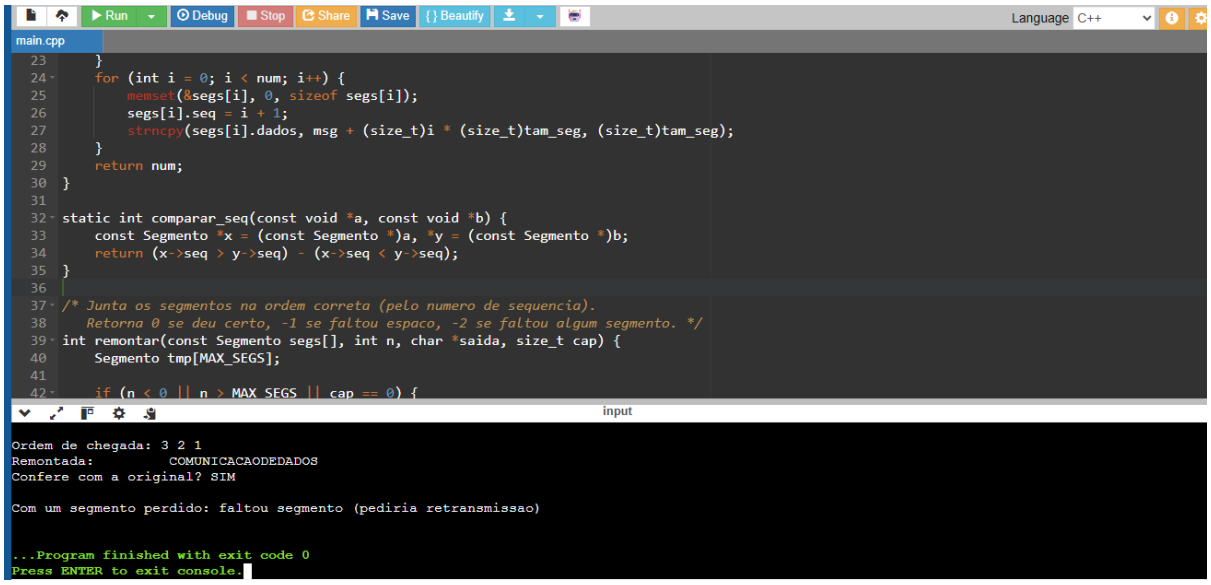
printf("\nOrdem de chegada:");
for (int i = 0; i < n; i++) {
    printf(" %d", segs[i].seq);
}
printf("\n");

if (remontar(segs, n, saida, sizeof saida) == 0) {
    printf("Remontada:         %s\n", saida);
```

```
    printf("Confere com a original? %s\n",
strcmp(msg, saida) == 0 ? "SIM" : "NAO");
}

/* simula perda: remove um segmento e tenta
remontar */
int r = remontar(segs, n - 1, saida, sizeof saida);
printf("\nCom um segmento perdido: %s\n",
       r == -2 ? "faltou segmento (pediria
retransmissao)" : "outro resultado");
return 0;


}
```




**C1 — Tamanho total do pacote**

Lê com `scanf` o tamanho dos dados e soma o cabeçalho fixo de 20 bytes, que simula o cabeçalho da camada de rede.
Esse é o **encapsulamento** do capítulo: tamanho total = cabeçalho + dados.
O programa recusa texto (abc) e números negativos. Usei `long long` para não estourar o inteiro com valores gigantes.
Com 100 bytes de dados, o total sai 120 bytes.

**C2 — Formatador de endereço físico**

A função recebe um vetor de 6 bytes (48 bits, o tamanho de um endereço Ethernet) e imprime cada byte em hexadecimal com 2 dígitos (`%02X`).
Coloca `:` entre os bytes, menos depois do último. O resultado é `07:01:02:01:2C:4B`, igual ao Exemplo 2.2 do slide.
Testei também com o endereço de broadcast, `FF:FF:FF:FF:FF:FF`.
Usei `unsigned char` para os bytes não ficarem com sinal.

**C3 — Classificador de camadas OSI**

O `enum` dá um número para cada camada, de 0 a 6.
O `switch` agrupa como no slide: física, enlace e rede são "suporte a rede". Transporte fica sozinha. Sessão, apresentação e aplicação são "suporte ao usuario".
Se vier um valor fora do enum, a função devolve "camada invalida".
O `main` imprime as 7 camadas numeradas de 1 a 7 com sua classificação.

**C4 — Corrigir a struct**

O erro era a ordem dos campos. Na struct original `dados` vinha antes de `cabecalho`, o contrário do encapsulamento (H | Dados).
Corrigi trocando a ordem e criei `montar_pacote`, que preenche primeiro o cabeçalho e depois os dados.
Troquei `strcpy` por `snprintf`, que nunca escreve além do tamanho do campo.
Também zerei a struct antes com `memset`.
O `offsetof` mostra a prova: o cabeçalho começa no byte 0 e os dados no byte 16.
A struct ocupa 144 bytes (16 do cabeçalho + 128 dos dados).

**C5 — Segmentação e remontagem**

`segmentar` divide a mensagem em pedaços de `tam_seg` bytes e dá a cada um um número de sequência. `COMUNICACAODEDADOS` com tamanho 8 vira `COMUNICA`, `CAODEDAD` e `OS`.
`remontar` ordena os segmentos pelo número de sequência (com `qsort`) e concatena. Por isso a mensagem volta certa mesmo se chegar fora de ordem.
No `main` simulei a chegada na ordem 3, 2, 1 e a mensagem foi reconstruída igual à original. Ela também confere se está faltando algum segmento: simulei uma perda e o programa detecta, como o TCP faria antes de pedir retransmissão.
Funções de remontagem como essa trabalham com uma cópia dos segmentos, então o vetor original não é alterado.